



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«Дальневосточный федеральный университет»
(ДФУ)

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ
(ШКОЛА)

Департамент математического и компьютерного моделирования

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

по образовательной программе подготовки бакалавров
по направлению 01.03.02 «Прикладная математика и информатика»
профиль «Математика и компьютерные технологии»

Универсальная система распознавания речи

Работа защищена
с оценкой _____

Студент группы № Б9122-01.03.02мкт

(подпись) Ляпин Д.А.
(Ф.И.О.)

«__» _____ 20__ г.

Регистрационный номер _____

Руководитель _____

(должность, учёное звание)

«__» _____ 20__ г.

(подпись) Акулов А.С.
(Ф.И.О.)

«__» _____ 20__ г.

г. Владивосток

2026

Содержание

Введение	4
Термины, определения и сокращения	8
1 Анализ предметной области и существующих решений	11
1.1 Задача транскрибации многоязычной речи	11
1.2 Обзор методов и моделей	12
1.2.1 Модели автоматического распознавания речи	12
1.2.2 Модели диаризации спикеров	14
1.2.3 Модели определения языка	15
1.3 Анализ существующих решений	16
1.4 Выводы по главе и требования к системе	17
2 Проектирование архитектуры системы	19
2.1 Общая архитектура системы	19
2.2 Архитектура pipeline обработки	20
2.2.1 Интерфейс этапа обработки	21
2.2.2 Движок pipeline	21
2.2.3 Контекст выполнения	21
2.2.4 Конкретные этапы pipeline	22
2.3 Модель данных	23
2.3.1 Внутренняя модель (уровень pipeline)	23
2.3.2 Внешняя модель (уровень API и результата)	23
2.3.3 Конфигурация	24
2.4 Система подключаемых модулей и реестр моделей	25
2.4.1 Абстрактные интерфейсы моделей	25
2.4.2 Реестр моделей	25
2.4.3 Механизм загрузки подключаемых модулей	26
2.5 Механизмы воспроизводимости и отказоустойчивости	27
2.5.1 Идентификатор запуска и структура хранения	27
2.5.2 Хранение промежуточных данных	27
2.5.3 Состояние pipeline и возобновление обработки	28
2.5.4 Стратегия обработки ошибок	28

2.6	Событийная модель и взаимодействие с графическим интерфейсом	29
2.6.1	Событийная шина	29
2.6.2	Адаптер для графического интерфейса	30
2.7	Проектирование графического интерфейса	30
2.7.1	Обоснование выбора PyQt6	30
2.7.2	Структура интерфейса	31
2.7.3	Сценарий использования	31
2.8	Выводы по главе	32

Список литературы	34
--------------------------	-----------

ВВЕДЕНИЕ

В условиях глобализации деловых коммуникаций задача точной и безопасной расшифровки многоязычных диалогов приобретает критическое значение для широкого спектра профессиональных сфер. Международные переговоры, судебные заседания, пресс-конференции и рабочие совещания с участием представителей различных языковых групп формируют постоянно растущий объем аудиоданных, требующих структурированной текстовой фиксации с разделением по спикерам и сохранением исходных языков высказываний.

Существующие технологические решения в области автоматического распознавания речи (Automatic Speech Recognition, ASR) не удовлетворяют в полной мере требованиям пользователей, формируя ряд фундаментальных проблем. Во-первых, наблюдается технологическая фрагментарность: специалисты вынуждены самостоятельно интегрировать разрозненные инструменты для диаризации (разделения речи по спикерам), определения языка (Language Identification, LID) и непосредственно транскрибации. Такой процесс требует глубоких технических компетенций и не гарантирует стабильного результата, особенно при работе с редкими языками или речью с выраженным акцентом.

Во-вторых, использование облачных сервисов распознавания речи сопряжено с рисками для конфиденциальности данных. Передача аудиозаписей переговоров, содержащих коммерческую, юридическую или журналистскую тайну, на серверы третьих сторон ставит под угрозу безопасность информации и может нарушать требования законодательства о защите персональных данных и внутренних политик организаций.

В-третьих, существующие решения для локального развертывания либо ограничены предустановленными моделями и не позволяют адаптироваться под специфические задачи пользователя, либо требуют дорогостоящего специализированного оборудования с графическими ускорителями (GPU), что делает их недоступными для большинства потенциальных пользователей. Таким образом, на рынке отсутствует универсальное, конфиденциальное и технологически доступное решение, объединяющее функции диаризации, определения языка и транскрибации в рамках единой расширяемой системы, способной функционировать на стандартном пользовательском оборудовании.

Указанные обстоятельства обуславливают актуальность разработки модульной программной системы, обеспечивающей полный цикл локальной обработки

многоязычных аудиозаписей с использованием pipeline-архитектуры и поддержкой подключаемых моделей машинного обучения.

Проблематика автоматического распознавания речи активно исследуется в работах отечественных и зарубежных учёных. Значительный вклад в развитие методов ASR внесли исследования, связанные с моделью Whisper [1], архитектурой Conformer [2] и подходами на основе CTC [3] и механизма внимания [4, 5]. Современные модели, такие как wav2vec 2.0 [6], продемонстрировали возможности самообучения на неразмеченных данных. Задачи диаризации спикеров рассмотрены в работах Н. Bredin и др. [7, 8, 9], а методы идентификации языка по аудиоданным — в исследованиях, посвящённых архитектуре ECAPA-TDNN [10] и набору данных VoxLingua107 [11]. Вместе с тем вопросы проектирования интегрированных модульных систем, объединяющих указанные компоненты в единый pipeline с поддержкой расширяемости через plugin-архитектуру [12, 13] и ориентированных на локальное развертывание без GPU, остаются недостаточно проработанными.

Цель работы — проектирование и реализация модульной программной системы оффлайн-транскрибации многоязычной речи с использованием pipeline-архитектуры, обеспечивающей полный цикл обработки аудиозаписей встреч (диаризация, определение языка, транскрибация) в локальном режиме на стандартном пользовательском оборудовании.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. провести анализ существующих методов и моделей автоматического распознавания речи [1, 2, 6], диаризации спикеров [7] и определения языка [11, 10], а также выполнить сравнительный обзор существующих программных решений в данной области;
2. сформулировать функциональные и нефункциональные требования к разрабатываемой системе;
3. спроектировать pipeline-архитектуру обработки аудиоданных [14, 15], обеспечивающую последовательное выполнение этапов диаризации, сегментации, определения языка и транскрибации;
4. разработать plugin-систему [12] для подключения и замены моделей распознавания речи без модификации основного кода системы;

5. реализовать механизмы воспроизводимости результатов обработки, включая сохранение конфигурации запусков, промежуточных данных и возможность возобновления прерванного pipeline;
6. реализовать графический интерфейс пользователя на базе PyQt6 [16], обеспечивающий загрузку аудиофайлов, запуск обработки и отображение результатов;
7. провести экспериментальную оценку качества работы системы на многоязычных аудиозаписях с использованием метрик WER [17] и DER [18].

Объектом исследования является процесс автоматической обработки многоязычных аудиозаписей с разделением речи по спикерам и определением языка высказываний.

Предметом исследования выступает программная архитектура модульной системы оффлайн-транскрибации, построенная на основе pipeline-обработки с поддержкой подключаемых моделей машинного обучения.

Методы исследования. В работе используются методы системного анализа и сравнительного анализа программных решений, объектно-ориентированного проектирования [12], а также паттерны архитектурного проектирования (Pipeline [14], Strategy, Plugin/Registry, Observer). Для оценки качества системы применяются методы экспериментального тестирования с использованием стандартных метрик оценки качества распознавания речи (WER) [17] и диаризации (DER) [18]. Тестирование проводится на открытых бенчмарках [19, 20].

Практическая значимость работы определяется тем, что разработанная система представляет собой готовое кроссплатформенное программное приложение, пригодное для использования в организациях, предъявляющих повышенные требования к конфиденциальности обрабатываемых данных. Система может применяться для протоколирования международных переговоров и совещаний, фиксации судебных заседаний и допросов, расшифровки интервью и пресс-конференций. Модульная архитектура обеспечивает возможность адаптации системы под специфические задачи пользователей путём подключения специализированных моделей распознавания речи.

Структура работы. Выпускная квалификационная работа состоит из введения, трёх глав, заключения, списка литературы и приложений.

В первой главе проводится анализ предметной области: формализуется задача транскрибации многоязычной речи, выполняется обзор существующих мо-

делей ASR [1, 2], диаризации [7] и определения языка [11], а также сравнительный анализ существующих программных решений. По результатам анализа формулируются требования к разрабатываемой системе.

Во второй главе описывается проектирование архитектуры системы: общая компонентная архитектура [13], pipeline-механизм обработки аудиоданных [14], модель данных, plugin-система и реестр моделей [12], механизмы воспроизводимости и отказоустойчивости, событийная модель взаимодействия с графическим интерфейсом.

В третьей главе представлены результаты реализации системы: описаны используемые технологии и инструменты [21, 22, 23, 16], ключевые аспекты реализации pipeline-ядра, plugin-системы и графического интерфейса. Приводятся результаты экспериментальной оценки качества работы системы на многоязычных аудиозаписях.

ТЕРМИНЫ, ОПРЕДЕЛЕНИЯ И СОКРАЩЕНИЯ

Термины и определения

Агломеративная кластеризация — метод иерархической кластеризации, при котором каждый объект изначально образует отдельный кластер; кластеры последовательно объединяются до достижения заданного числа групп или порогового значения расстояния.

Диаризация спикеров (speaker diarization) — задача автоматического определения временных границ речевых фрагментов аудиозаписи и атрибуции каждого фрагмента конкретному говорящему (ответ на вопрос «кто говорил и когда»).

Инференс — применение обученной нейросетевой модели к входным данным с целью получения предсказания; в контексте работы: распознавание речи или определение языка.

Квантизация модели — метод оптимизации нейросетевых моделей, при котором числовые представления параметров переводятся в формат меньшей разрядности, уменьшая размер модели и ускоряя инференс при незначительной потере точности.

Конвейер обработки (pipeline) — архитектурный паттерн, при котором данные последовательно проходят через цепочку независимых этапов; выход каждого этапа является входом следующего.

Маршрутизация модели — динамический выбор конкретной модели обработки на основании атрибутов входных данных; в контексте работы — выбор ASR-модели в зависимости от определённого языка сегмента.

Подключаемый модуль (plugin) — независимый программный компонент, расширяющий функциональность системы через заранее определённый интерфейс без изменения основного кода системы.

Реестр моделей (Model Registry) — программный компонент, реализующий паттерн «Реестр» и предоставляющий единую именованную точку доступа к зарегистрированным моделям машинного обучения.

Сегментация аудио — разделение исходного аудиофайла на отдельные фрагменты в соответствии с временной разметкой, полученной на этапе диаризации.

Событийная шина (Event Bus) — программный компонент, реализующий паттерн «Наблюдатель»; обеспечивает слабосвязанное взаимодействие между компонентами системы путём публикации и подписки на типизированные события.

Транскрибация речи — автоматическое преобразование речевого аудиосигнала в текстовую форму с помощью модели распознавания речи.

Эмбединг (векторное представление) — компактное числовое представление объекта (аудиосигнала, спикера, языкового фрагмента) в многомерном пространстве признаков, порождённое нейросетевым кодировщиком.

Перечень сокращений

ОЗУ	Оперативное запоминающее устройство
ПК	Персональный компьютер
API	Application Programming Interface — программный интерфейс приложения
ASR	Automatic Speech Recognition — автоматическое распознавание речи
CPU	Central Processing Unit — центральный процессор
CTC	Connectionist Temporal Classification — метод обучения нейронных сетей для разметки последовательностей без выравнивания
DER	Diarization Error Rate — частота ошибок диаризации
ECAPA-TDNN	Emphasized Channel Attention, Propagation and Aggregation in Time Delay Neural Network — архитектура нейронной сети для распознавания спикера и определения языка
GPU	Graphics Processing Unit — графический процессор
GUI	Graphical User Interface — графический интерфейс пользователя
JSON	JavaScript Object Notation — текстовый формат обмена структурированными данными
LID	Language Identification — определение языка по аудиосигналу
TDNN	Time Delay Neural Network — нейронная сеть с задержкой по времени
VAD	Voice Activity Detection — определение голосовой активности
WER	Word Error Rate — частота ошибок распознавания на уровне слов
YAML	YAML Ain't Markup Language — формат сериализации данных, используемый для конфигурационных файлов

1 Анализ предметной области и существующих решений

1.1 Задача транскрибации многоязычной речи

Задача, решаемая в рамках настоящей работы, может быть сформулирована следующим образом. На вход системе подаётся аудиофайл записи встречи, содержащий речь нескольких спикеров на разных языках. На выходе система должна сформировать структурированную текстовую расшифровку, в которой каждому высказыванию сопоставлены идентификатор спикера, временные метки начала и конца, определённый язык и распознанный текст.

Данная задача естественным образом декомпозируется на четыре последовательных подзадачи:

1. *Диаризация спикеров* (speaker diarization) — определение временных границ речевых фрагментов и их атрибуция конкретным спикерам, то есть ответ на вопрос «кто говорил и когда».
2. *Сегментация аудио* — разделение исходного аудиофайла на отдельные аудиофрагменты в соответствии с полученной разметкой диаризации.
3. *Определение языка* (Language Identification, LID) — классификация каждого аудиофрагмента по языку произнесённой речи.
4. *Транскрибация* (Automatic Speech Recognition, ASR) — преобразование каждого аудиофрагмента в текст с использованием модели, соответствующей определённому языку.

Последовательный характер указанных подзадач определяет выбор pipeline-архитектуры в качестве основного архитектурного паттерна системы [14].

К разрабатываемой системе предъявляется ряд ключевых требований, обусловленных предметной областью:

- *Локальность обработки*: все данные должны обрабатываться на оборудовании пользователя без передачи в облачные сервисы, что исключает риски утечки конфиденциальной информации.
- *Работа на стандартном оборудовании*: система должна функционировать на ПК с 16 Гб оперативной памяти без необходимости наличия GPU. Допустимое время обработки одного часа аудио составляет до 24 часов.

- *Мультиязычность*: поддержка нескольких языков в рамках одной записи, включая языки с ограниченными ресурсами (low-resource languages).
- *Расширяемость*: возможность подключения и замены моделей распознавания без модификации основного кода.

Для количественной оценки качества работы системы используются стандартные метрики. Для оценки качества распознавания речи применяется метрика WER (Word Error Rate) — отношение числа ошибочно вставленных, удалённых и заменённых слов к общему числу слов в эталонной расшифровке [17]. Для оценки качества диаризации используется метрика DER (Diarization Error Rate), учитывающая три типа ошибок: пропущенная речь (missed speech), ложное обнаружение речи (false alarm) и ошибка атрибуции спикера (speaker confusion) [18]. Для оценки точности определения языка применяется показатель error rate на уровне сегментов.

1.2 Обзор методов и моделей

1.2.1 Модели автоматического распознавания речи

Область автоматического распознавания речи переживает период интенсивного развития, связанный с переходом от классических гибридных систем на основе скрытых марковских моделей к сквозным нейросетевым архитектурам. В настоящем обзоре рассматриваются модели, наиболее релевантные для задачи мультиязычной оффлайн-транскрибации.

Whisper. Модель Whisper, представленная компанией OpenAI [1], является сквозной моделью типа encoder-decoder на основе архитектуры Transformer [5]. Whisper обучена на более чем 680 000 часов слабо размеченных мультиязычных аудиоданных и поддерживает распознавание речи на 99 языках, а также перевод речи и определение языка. На бенчмарке LibriSpeech [19] модель Whisper large-v3 демонстрирует WER порядка 2,7% на чистой речи и около 5,2% в сложных акустических условиях. На мультиязычном бенчмарке Common Voice 15 средний WER составляет около 10% по всем поддерживаемым языкам. Модель распространяется под лицензией MIT.

Ключевой особенностью Whisper является мультизадачная архитектура: единая модель выполняет распознавание речи, перевод, определение языка и обнаружение голосовой активности, что позволяет заменить несколько компонентов традиционного речевого pipeline [1].

Для повышения скорости инференса на CPU разработаны оптимизированные реализации: `faster-whisper` [22], использующая движок `CTranslate2` для ускорения до 4 раз при сохранении точности, и `whisper.cpp` [24] — порт модели на C/C++ с поддержкой квантизации.

Conformer. Архитектура Conformer [2], предложенная исследователями из Google, объединяет механизм самовнимания (self-attention) из Transformer с одномерными свёрточными слоями для совместного моделирования глобальных и локальных зависимостей аудиосигнала. На бенчмарке LibriSpeech Conformer достигает WER 1,9% / 3,9% (test-clean / test-other) с внешней языковой моделью. Архитектура Conformer лежит в основе многих современных ASR-систем, включая NVIDIA NeMo и Google USM.

wav2vec 2.0. Модель wav2vec 2.0 [6] реализует подход самообучения (self-supervised learning): энкодер предварительно обучается на неразмеченных аудиоданных посредством контрастной задачи, а затем дообучается на небольшом объёме размеченных данных. Данный подход особенно эффективен для языков с ограниченными ресурсами, где объём размеченных данных невелик.

Vosk. Vosk [25] — офлайн-библиотека распознавания речи, поддерживающая около 20 языков. Vosk основан на фреймворке Kaldi и предоставляет компактные модели, пригодные для работы на CPU. Основным преимуществом является малый размер моделей (50–300 МБ) и низкие требования к ресурсам, однако точность распознавания уступает моделям класса Whisper, особенно на мультиязычных данных.

В таблице 1 приведено сравнение рассмотренных ASR-моделей по ключевым характеристикам.

Таблица 1 – Сравнение моделей автоматического распознавания речи

Модель	Языки	WER (en)	Размер	CPU	Лицензия
Whisper large-v3	99	2,7%	1,55 ГБ	да*	MIT
Conformer (L)	зав. от обуч.	1,9%	~120М	зав. от реал.	Apache 2.0
wav2vec 2.0	зав. от обуч.	1,8%	~300М	да	MIT
Vosk	~20	5–10%	50–300 МБ	да	Apache 2.0

* — через оптимизированные реализации (faster-whisper, whisper.cpp)

1.2.2 Модели диаризации спикеров

Диаризация спикеров (speaker diarization) — задача автоматического определения временных границ речевых фрагментов и их атрибуции конкретным спикерам [18]. Современные подходы к диаризации можно разделить на два основных направления: модульные pipeline-системы и сквозные (end-to-end) методы.

pyannote.audio. Библиотека pyannote.audio [7, 8] представляет собой открытый инструментарий на Python для задач диаризации спикеров. Версия 3.1 pipeline диаризации включает три основных этапа:

1. *Сегментация спикеров*: нейросетевая модель сегментации применяется к скользящему окну аудио и определяет активность каждого спикера на локальном уровне. Модель обучена с использованием Powerset multi-class cross-entropy loss [9].
2. *Извлечение эмбедингов*: для каждого локально определённого спикера извлекается нейросетевой эмбединг.
3. *Агломеративная кластеризация*: локальные эмбединги объединяются в глобальные кластеры, каждый из которых соответствует одному спикеру.

Pipeline pyannote 3.1 демонстрирует DER порядка 18,8% на датасете AMI (IHM), 12,2% на AISHELL-4 и 21,4% на DIHARD 3 в наиболее строгих условиях оценки (без forgiveness collar, с учётом перекрывающейся речи). Pipeline обучен на комбинации датасетов AISHELL, AliMeeting, AMI, AVA-AVD, DIHARD, Ego4D, MSDWild, REPERE и VoxConverse. Обработка одного часа аудио на GPU (NVIDIA V100) занимает около 1,5 минуты; на CPU время обработки существенно возрастает, однако библиотека полностью поддерживает CPU-инференс.

NVIDIA NeMo. Фреймворк NVIDIA NeMo предоставляет альтернативный pipeline диаризации, основанный на модели TitaNet [26] для извлечения эмбеддингов и кластеризации. NeMo ориентирован преимущественно на GPU-инференс, что ограничивает его применимость в контексте требований настоящей работы.

Обоснованием выбора ruannote.audio в качестве основного инструмента диаризации служат: открытый исходный код, поддержка CPU-инференса, высокое качество на стандартных бенчмарках, модульная архитектура и активное сообщество разработчиков.

1.2.3 Модели определения языка

Задача определения языка по аудиоданным (Spoken Language Identification, LID) заключается в классификации аудиофрагмента по языку произнесённой речи.

SpeechBrain / ECAPA-TDNN. Модель speechbrain/lang-id-voxlangua107-ecapa [21] представляет собой классификатор языка, обученный на наборе данных VoxLingua107 [22] с использованием архитектуры ECAPA-TDNN [10]. Архитектура ECAPA-TDNN расширяет классическую TDNN (Time Delay Neural Network) механизмами канального внимания (Squeeze-and-Excitation), многоуровневой агрегацией признаков и улучшенным статистическим пулингом. Набор данных VoxLingua107 содержит 6628 часов речи на 107 языках, собранных из видеозаписей YouTube. Модель демонстрирует уровень ошибок 6,7% на валидационном множестве VoxLingua107.

Whisper LID. Модель Whisper [1] обладает встроенной функцией определения языка: декодер модели включает специальные токены языковых идентификаторов, позволяющие классифицировать язык на этапе инференса. Точность определения языка Whisper коррелирует с объёмом обучающих данных для конкретного языка и составляет 90–99% для высокоресурсных языков.

В контексте настоящей работы преимуществом модели SpeechBrain является её специализация на задаче LID, более широкий охват языков (107 языков) и возможность использования независимо от ASR-модели, что соответствует принципу модульности разрабатываемой системы.

1.3 Анализ существующих решений

Существующие решения для транскрибации многоязычной речи можно разделить на две категории: облачные сервисы и локальные инструменты.

Облачные сервисы. К наиболее распространённым облачным решениям относятся:

- *Яндекс SpeechKit* — облачный сервис распознавания и синтеза речи, поддерживающий русский, английский, немецкий, французский и другие языки. Предоставляет API для потоковой и пакетной обработки, а также функцию диаризации. Требуется передачи данных в облако Яндекса.
- *Сбер SaluteSpeech* — платформа распознавания речи от Сбера, ориентированная на русский язык. Обеспечивает высокое качество распознавания русской речи, однако мультязычные возможности ограничены.
- *Google Cloud Speech-to-Text* — облачный сервис, поддерживающий более 125 языков, с функциями диаризации и автоматического определения языка. Является одним из наиболее функциональных решений, однако требует передачи данных в облако Google.
- *Stenogram.ai* — специализированный сервис для расшифровки аудиозаписей встреч с поддержкой диаризации и мультязычности.

Ключевым недостатком всех облачных решений является необходимость передачи аудиоданных на удалённые серверы, что создаёт неприемлемые риски для обработки конфиденциальных записей.

Локальные инструменты. Среди локальных решений можно выделить:

- *whisper.cpp* [24] — реализация модели Whisper на C/C++ с поддержкой CPU-инференса и квантизации. Предоставляет только функцию транскрибации, без диаризации и мультязычной маршрутизации моделей. Интерфейс ограничен командной строкой.

- *Vosk Server* [25] — серверная реализация офлайн-распознавания на базе Kaldi. Поддерживает около 20 языков, однако не предоставляет диаризацию и определение языка. Модели предустановлены и не допускают замену пользователем.
- *faster-whisper* [22] — оптимизированная реализация Whisper на основе CTranslate с ускорением до 4 раз. Как и *whisper.cpp*, является исключительно ASR-инструментом без интеграции с диаризацией и LID.

В таблице 2 приведено сравнение существующих решений по ключевым критериям, релевантным для задачи настоящей работы.

Таблица 2 – Сравнение существующих решений для транскрибации речи

Решение	Локальн.	Мультияз.	Диариз.	Расшир.	GUI	CPU
Яндекс SpeechKit	нет	частично	да	нет	нет	—
Сбер SaluteSpeech	нет	частично	да	нет	нет	—
Google Cloud STT	нет	да	да	нет	нет	—
Stenogram.ai	нет	да	да	нет	да	—
whisper.cpp	да	да	нет	нет	нет	да
Vosk Server	да	частично	нет	нет	нет	да
faster-whisper	да	да	нет	нет	нет	да
Разраб. система	да	да	да	да	да	да

Анализ показывает, что ни одно из существующих решений не объединяет все требуемые характеристики: локальность обработки, полную мультиязычность с маршрутизацией ASR-моделей по языку, диаризацию, расширяемость через plugin-систему, графический интерфейс и работу на CPU. Облачные сервисы решают задачу комплексно, но исключают локальную обработку. Локальные инструменты предоставляют лишь отдельные компоненты (как правило, только ASR), требуя от пользователя самостоятельной интеграции.

1.4 Выводы по главе и требования к системе

На основании проведённого анализа предметной области, существующих моделей и программных решений можно сформулировать следующие выводы:

1. Задача транскрибации многоязычной речи с разделением по спикерам де-

композируется на четыре подзадачи (диаризация, сегментация, LID, ASR), каждая из которых решается специализированными моделями машинного обучения.

2. Современные модели ASR (Whisper [1]), диаризации (pyannote.audio [7]) и определения языка (ECAPA-TDNN на VoxLingua107 [11, 10]) достигли уровня качества, достаточного для практического применения.
3. Существующие решения не объединяют указанные компоненты в единую локальную систему с поддержкой расширяемости и графическим интерфейсом.

На основании анализа сформулированы следующие **функциональные требования** к разрабатываемой системе:

1. загрузка аудиофайлов распространённых форматов (WAV, MP3, FLAC, OGG);
2. автоматическая диаризация спикеров с использованием pyannote.audio [7];
3. сегментация аудиозаписи на отдельные фрагменты в соответствии с разметкой диаризации;
4. определение языка каждого аудиофрагмента с использованием модели LID [11];
5. транскрибация каждого фрагмента с автоматическим выбором ASR-модели в зависимости от определённого языка;
6. формирование итоговой расшифровки, структурированной по спикерам с метками языка и временными метками;
7. экспорт результатов в форматы JSON и TXT;
8. графический интерфейс с возможностью загрузки файла, запуска обработки, отображения прогресса и просмотра результатов.

Нефункциональные требования к разрабатываемой системе:

1. полностью локальная обработка без передачи данных в облачные сервисы;
2. работа на стандартном оборудовании: CPU, 16 ГБ ОЗУ, без GPU;
3. время обработки одного часа аудио не должно превышать 24 часов на CPU;

4. кроссплатформенность: поддержка операционных систем Windows, Linux, macOS;
5. воспроизводимость: сохранение конфигурации каждого запуска, промежуточных результатов и возможность возобновления прерванной обработки;
6. модульность: возможность подключения и замены моделей ASR, LID и диаризации через систему подключаемых модулей без модификации основного кода [12].

2 Проектирование архитектуры системы

2.1 Общая архитектура системы

Архитектура разрабатываемой системы построена на принципе разделения ответственности и включает четыре основных архитектурных слоя [13]:

1. *core* — ядро системы, содержащее всю логику обработки аудиоданных: pipeline-движок, абстрактные интерфейсы моделей, реестр моделей, хранилище данных и событийную шину. Ядро не зависит от конкретных реализаций моделей и графического интерфейса;
2. *plugins* — подключаемые модули, реализующие конкретные модели машинного обучения (ASR, диаризация, определение языка) через интерфейсы, определённые в ядре;
3. *infrastructure* — инфраструктурный слой, обеспечивающий взаимодействие с SQLite, файловой системой и системой логирования;
4. *ui* — слой графического интерфейса на основе PyQt6 [16], взаимодействующий с ядром исключительно через программный интерфейс и событийную шину.

Принципиальным архитектурным решением является строгое разделение ядра (*core*) и интерфейса (*ui*). Логика обработки речи полностью инкапсулирована в ядре, которое может использоваться как самостоятельная Python-библиотека в сторонних проектах и скриптах. Графический интерфейс является лишь одним из возможных клиентов ядра.

Для обеспечения гибкости использования реализован двухуровневый программный интерфейс. Основной интерфейс представлен объектом `Pipeline`, который управляет полным жизненным циклом обработки: запуском, приостановкой, возобновлением и получением текущего состояния. Объект `Pipeline` поддерживает работу с идентификаторами запусков и промежуточными результатами. Дополнительно предоставляется упрощённый высокоуровневый метод `transcribe(audio_path, config)`, реализованный как обёртка над `Pipeline` и предназначенный для сценариев, не требующих детального контроля над процессом обработки.

Файловая структура проекта организована в соответствии с описанными архитектурными слоями:

```
project/
├── core/
│   ├── pipeline/          # Движок pipeline и этапы обработки
│   ├── models/            # Абстрактные интерфейсы моделей
│   ├── registry/          # Реестр моделей
│   ├── storage/           # Хранилище данных и управление запусками
│   ├── events/            # Событийная шина
│   ├── config/            # Конфигурация
│   └── api/               # Программный интерфейс
├── plugins/               # Подключаемые модули моделей
├── infrastructure/        # SQLite, файловая система, логирование
├── ui/                    # Графический интерфейс (PyQt6)
└── configs/               # Файлы конфигурации YAML
```

2.2 Архитектура pipeline обработки

Обработка аудиоданных реализована в виде последовательности независимых этапов (stages), каждый из которых принимает данные предыдущего этапа, выполняет преобразование и передаёт результат следующему [14]. Такая архитектура соответствует паттерну «Pipes and Filters» и обеспечивает возможность независимого тестирования, замены и расширения отдельных этапов [15].

2.2.1 Интерфейс этапа обработки

Каждый этап pipeline реализует единый интерфейс **Stage**, определяющий контракт взаимодействия с движком:

```
class Stage(ABC):
    name: str

    @abstractmethod
    def run(self, segments: List[Segment],
            context: PipelineContext) -> List[Segment]:
        ...
```

Метод `run` принимает список сегментов и контекст выполнения, выполняет преобразование и возвращает обновлённый список сегментов. Интерфейс не содержит зависимостей от конкретных моделей или внешних библиотек, что обеспечивает возможность подмены реализации.

2.2.2 Движок pipeline

Класс **PipelineEngine** отвечает за последовательный запуск этапов, передачу контекста между ними, сохранение промежуточных результатов после каждого этапа и публикацию событий через событийную шину. Движок реализует следующую последовательность действий для каждого этапа:

1. публикация события **StageStarted**;
2. вызов метода `run` текущего этапа;
3. сохранение результата (списка сегментов) в хранилище;
4. обновление состояния pipeline (фиксация завершённого этапа);
5. публикация события **StageCompleted**.

2.2.3 Контекст выполнения

Объект **PipelineContext** инкапсулирует данные, необходимые для работы этапов, но не являющиеся частью потока сегментов: путь к исходному аудио-

файлу, конфигурацию запуска, ссылку на реестр моделей и событийную шину. Контекст создаётся при инициализации pipeline и передаётся каждому этапу без изменений.

2.2.4 Конкретные этапы pipeline

Pipeline включает четыре основных этапа обработки:

1. *DiarizationStage* — выполняет диаризацию исходного аудиофайла с использованием модели, полученной из реестра. На выходе формирует список сегментов с заполненными полями временных меток (`start_time`, `end_time`) и идентификатора спикера (`speaker_id`);
2. *SegmentationStage* — разделяет исходный аудиофайл на отдельные аудиофрагменты в соответствии с временными метками, полученными на этапе диаризации. Каждому сегменту сопоставляется путь к соответствующему аудиофрагменту на диске;
3. *LanguageIdentificationStage* — определяет язык речи в каждом аудиофрагменте с помощью модели LID из реестра. Результат записывается в поле `language` каждого сегмента;
4. *TranscriptionStage* — выполняет распознавание речи для каждого сегмента. На основании определённого языка сегмента этап запрашивает из реестра соответствующую ASR-модель и вызывает метод транскрибации. Результат записывается в поле `text` сегмента.

Маршрутизация ASR-модели по языку на этапе транскрибации реализована через обращение к конфигурации, содержащей маппинг «язык → идентификатор модели»:

```
model = registry.get("asr", config.asr_models[segment.language])
segment.text = model.transcribe(segment.audio_path)
```

Общий pipeline реализован как синхронная последовательность этапов. При этом внутри отдельных этапов (определение языка и транскрибация) допускается параллельная обработка сегментов через локальную систему задач, что позволяет эффективно использовать ресурсы без избыточной сложности.

2.3 Модель данных

В системе используется гибридная модель данных, учитывающая два различных контекста использования.

2.3.1 Внутренняя модель (уровень pipeline)

На уровне pipeline данные представлены как плоский список независимых сегментов. Каждый сегмент описывается структурой `Segment`, содержащей следующие поля:

```
@dataclass
class Segment:
    start_time: float      # Время начала (секунды)
    end_time: float        # Время конца (секунды)
    speaker_id: str = ""   # Идентификатор спикера
    language: str = ""     # Код языка (ISO 639-1)
    text: str = ""         # Распознанный текст
    audio_path: str = ""   # Путь к аудиофрагменту
    confidence: float = 0.0 # Уверенность модели
```

Плоская структура обеспечивает простоту реализации этапов обработки: каждый этап получает список сегментов, обогащает их дополнительными данными и возвращает обновлённый список. Этапы не зависят друг от друга и могут обрабатывать сегменты независимо.

2.3.2 Внешняя модель (уровень API и результата)

На уровне программного интерфейса и итогового результата данные представлены как иерархическая структура встречи:

```
@dataclass
class Meeting:
    audio_path: str
    duration: float
    speakers: List[Speaker]
```

```
@dataclass
class Speaker:
    id: str
    segments: List[Segment]
```

Преобразование из плоского списка сегментов в иерархическую структуру выполняется на этапе формирования результата путём группировки сегментов по идентификатору спикера.

2.3.3 Конфигурация

Конфигурация системы реализована с использованием гибридного подхода. Внутренняя конфигурация представлена строго типизированной структурой `Config`, используемой pipeline:

```
@dataclass
class Config:
    diarization_model: str          # ID модели диаризации
    lid_model: str                  # ID модели определения языка
    asr_models: Dict[str, str]      # Маппинг: язык -> ID модели ASR
    output_format: str              # Формат вывода (json, txt)
    device: str                     # Устройство (cpu, cuda)
```

Для взаимодействия с пользователем конфигурация формируется автоматически через графический интерфейс. Дополнительно поддерживается загрузка и сохранение конфигурации в формате YAML для ручного редактирования и воспроизводимости запусков:

```
diarization_model: "pyannote/speaker-diarization-3.1"
lid_model: "speechbrain/lang-id-voxl107-ecapa"
asr_models:
  en: "openai/whisper-large-v3"
  ru: "bond005/whisper-large-v3-ru"
  zh: "openai/whisper-large-v3"
device: "cpu"
```


2.4 Система подключаемых модулей и реестр моделей

Для обеспечения расширяемости системы без модификации основного кода применяется архитектура подключаемых модулей [12], включающая два взаимодополняющих механизма: абстрактные интерфейсы моделей и реестр моделей.

2.4.1 Абстрактные интерфейсы моделей

Для каждого типа моделей определён абстрактный интерфейс, задающий контракт взаимодействия с pipeline. Паттерн «Стратегия» (Strategy) [12] позволяет подменять конкретные реализации моделей без изменения кода этапов обработки.

```
class ASRModel(ABC):
    @abstractmethod
    def transcribe(self, audio_path: str) -> str: ...

    @abstractmethod
    def get_supported_languages(self) -> List[str]: ...

class DiarizationModel(ABC):
    @abstractmethod
    def diarize(self, audio_path: str) -> List[Segment]: ...

class LanguageModel(ABC):
    @abstractmethod
    def detect(self, audio_path: str) -> str: ...
```

Каждый подключаемый модуль реализует один из указанных интерфейсов и обязан декларировать метаданные: тип модели, поддерживаемые языки и требования к ресурсам. Метаданные используются реестром моделей и графическим интерфейсом для корректного отображения и выбора моделей.

2.4.2 Реестр моделей

Класс `ModelRegistry` реализует паттерн «Реестр» (Registry) и обеспечивает единую точку доступа к моделям из pipeline:

```

class ModelRegistry:
    def register(self, model_type: str,
                 model_id: str,
                 model_class: Type) -> None: ...

    def get(self, model_type: str,
            model_id: str) -> Union[ASRModel,
                                    DiarizationModel,
                                    LanguageModel]: ...

    def list_models(self,
                    model_type: str) -> List[ModelInfo]: ...

```

Реестр поддерживает два способа указания моделей: через идентификатор HuggingFace (например, "openai/whisper-large-v3") и через локальный путь на диске. Это позволяет использовать как публичные предобученные модели, так и собственные дообученные версии пользователя.

2.4.3 Механизм загрузки подключаемых модулей

Основным механизмом подключения модулей является директория `plugins/`, из которой система динамически загружает модули, реализующие интерфейсы моделей. При запуске системы выполняется сканирование директории, обнаруженные модули регистрируются в реестре. Архитектура предусматривает возможность расширения через механизм Python `entrypoints` (pip-пакеты с объявленными точками входа), что позволит в будущем подключать внешние модули без размещения файлов в директории `plugins/`.

Процедура подключения нового модуля сводится к следующим действиям:

1. создание класса, реализующего соответствующий абстрактный интерфейс (`ASRModel`, `DiarizationModel` или `LanguageModel`);
2. заполнение метаданных модуля (тип, языки, требования);
3. размещение файла модуля в директории `plugins/`.

Изменения основного кода системы при этом не требуются, что соответствует принципу открытости/закрытости (Open/Closed Principle) [13].

2.5 Механизмы воспроизводимости и отказоустойчивости

Длительность обработки аудиозаписи (до 24 часов на CPU) обуславливает необходимость механизмов сохранения состояния, воспроизводимости результатов и устойчивости к сбоям.

2.5.1 Идентификатор запуска и структура хранения

Каждому запуску обработки присваивается уникальный идентификатор (`run_id`), под которым организуется хранение всех связанных данных. Файловая структура запуска имеет следующий вид:

```
runs/
└─ run_2025-06-15_143022/
   ├── config.yaml           # Конфигурация запуска
   ├── pipeline_state.json   # Состояние pipeline
   ├── stage_1_diarization/  # Результат диаризации
   ├── stage_2_segmentation/ # Аудиофрагменты
   ├── stage_3_lid/          # Результат определения языка
   ├── stage_4_transcription/ # Результат транскрибации
   └─ result.json            # Итоговый результат
```

Управление запусками осуществляется классом `RunManager`, отвечающим за создание директории запуска, присвоение идентификатора, сохранение конфигурации и предоставление доступа к данным предыдущих запусков.

2.5.2 Хранение промежуточных данных

Для хранения промежуточных данных применяется гибридный подход. Метаданные и состояние `pipeline` хранятся в базе данных SQLite [27], обеспечивающей структурированность данных, возможность выполнения запросов и атомарность операций записи. Аудиофрагменты и другие объёмные данные хранятся непосредственно в файловой системе, что обеспечивает эффективность работы с большими файлами.

Результаты каждого этапа `pipeline` сохраняются на диск сразу после его завершения. Это позволяет использовать их повторно при перезапуске: если при

повторном запуске с той же конфигурацией обнаруживаются сохранённые результаты этапа, он пропускается, что фактически реализует механизм кэширования.

2.5.3 Состояние pipeline и возобновление обработки

Класс `PipelineState` отслеживает текущее состояние обработки: список завершённых этапов, временные метки начала и окончания каждого этапа, информацию об ошибках. Состояние сохраняется в файл `pipeline_state.json` после завершения каждого этапа.

При вызове метода `resume(run_id)` движок pipeline определяет последний успешно завершённый этап, загружает его результаты из хранилища и продолжает обработку со следующего этапа. Это позволяет возобновить обработку после сбоя, перезагрузки системы или намеренного прерывания без потери уже выполненной работы.

2.5.4 Стратегия обработки ошибок

В системе применяется комбинированная стратегия обработки ошибок, учитывающая различные уровни критичности:

1. *временные технические сбои* (например, ошибка выделения памяти, временная недоступность файла) — применяется ограниченное число повторных попыток (retry) с экспоненциальной задержкой;
2. *ошибки на уровне отдельных сегментов* (например, неудачная транскрибация одного фрагмента из-за шума) — сегмент пропускается (skip) с сохранением информации об ошибке, обработка остальных сегментов продолжается. Частичный результат сохраняется;
3. *критические ошибки* (некорректная конфигурация, отсутствие модели, невозможность выполнения ключевого этапа) — выполнение pipeline останавливается с фиксацией текущего состояния и подробным логированием. Пользователь получает уведомление об ошибке.

2.6 Событийная модель и взаимодействие с графическим интерфейсом

Взаимодействие между ядром системы и графическим интерфейсом реализовано через событийную модель, обеспечивающую слабую связность компонентов [12].

2.6.1 Событийная шина

В ядре системы реализована событийная шина (**EventBus**), реализующая паттерн «Наблюдатель» (**Observer**) [12]. Pipeline публикует события через шину, не зная о конкретных подписчиках:

```
class EventBus:
    def subscribe(self, event_type: Type[Event],
                  handler: Callable) -> None: ...

    def publish(self, event: Event) -> None: ...
```

Определены следующие типы событий:

1. **StageStarted** — начало выполнения этапа (содержит имя этапа);
2. **StageCompleted** — завершение этапа (содержит имя этапа и длительность);
3. **ProgressUpdated** — обновление прогресса внутри этапа (содержит процент выполнения и описание текущей операции);
4. **ErrorOccurred** — возникновение ошибки (содержит тип ошибки, описание и информацию о сегменте);
5. **PipelineCompleted** — завершение всего pipeline (содержит путь к результату).

Событийная шина не содержит зависимостей от графического интерфейса и может использоваться для любых целей: логирование, отправка уведомлений, отображение прогресса.

2.6.2 Адаптер для графического интерфейса

В слое интерфейса реализуется адаптер, подписывающийся на события шины и преобразующий их в механизм сигналов Qt (`pyqtSignal`). Такой подход решает проблему потокобезопасности: `pipeline` выполняется в фоновом потоке (`QThread`), а обновление элементов интерфейса — в основном потоке Qt через механизм сигналов и слотов:

```
class QtEventAdapter(QObject):
    progress_updated = pyqtSignal(float, str)
    stage_started = pyqtSignal(str)
    error_occurred = pyqtSignal(str)

    def __init__(self, event_bus: EventBus):
        super().__init__()
        event_bus.subscribe(ProgressUpdated,
                           self._on_progress)
        event_bus.subscribe(StageStarted,
                           self._on_stage_started)
```

2.7 Проектирование графического интерфейса

2.7.1 Обоснование выбора PyQt6

В качестве фреймворка для графического интерфейса выбран PyQt6 [16]. Выбор обусловлен следующими факторами:

1. кроссплатформенность: PyQt6 поддерживает Windows, Linux и macOS, что соответствует требованию кроссплатформенности системы;
2. зрелость и стабильность: библиотека Qt используется в промышленной разработке более 25 лет и обладает обширной документацией;
3. поддержка многопоточности: встроенный механизм сигналов и слотов обеспечивает потокобезопасное взаимодействие фонового `pipeline` с элементами интерфейса;

4. совместимость с PyInstaller [28]: приложение на PyQt6 может быть собрано в автономный исполняемый файл для распространения пользователям, не знакомым с программированием.

2.7.2 Структура интерфейса

Графический интерфейс первой версии реализует минимальный, но функционально достаточный набор экранов. Главное окно приложения содержит следующие функциональные области:

1. *область загрузки файла* — позволяет выбрать аудиофайл через стандартный диалог операционной системы. Отображает имя выбранного файла, его длительность и формат;
2. *область настроек* — предоставляет возможность выбора моделей диаризации, определения языка и транскрибации из списка доступных в реестре. Также позволяет задать маппинг языков к ASR-моделям;
3. *область прогресса* — отображает текущий этап обработки, индикатор прогресса (progress bar) и текстовый журнал операций;
4. *область результатов* — отображает итоговую расшифровку, структурированную по спикерам, с возможностью экспорта в файл.

2.7.3 Сценарий использования

Основной сценарий использования приложения включает следующую последовательность действий пользователя:

1. пользователь запускает приложение и выбирает аудиофайл записи встречи;
2. при необходимости пользователь настраивает параметры обработки (выбор моделей, маппинг языков);
3. пользователь нажимает кнопку «Запустить обработку»;
4. система отображает прогресс обработки: текущий этап, процент выполнения, журнал операций;

5. по завершении обработки система отображает результат и предоставляет возможность экспорта в файл;
6. в случае ошибки система отображает уведомление с описанием проблемы и предлагает возобновить обработку.

В будущих версиях интерфейс планируется расширить до полноценного редактора: графическое отображение аудиофрагментов с цветовой маркировкой спикеров и языков, возможность воспроизведения отдельных сегментов, редактирование распознанного текста в режиме реального времени. Однако данная функциональность выходит за рамки первой версии системы.

2.8 Выводы по главе

В настоящей главе описано проектирование архитектуры системы оффлайн-транскрибации многоязычной речи. Основные проектные решения включают:

1. четырёхслойную архитектуру с разделением на ядро, подключаемые модули, инфраструктуру и интерфейс, обеспечивающую независимость компонентов и возможность использования ядра как библиотеки;
2. pipeline-архитектуру обработки с единым интерфейсом этапов, позволяющую расширять и модифицировать последовательность обработки без изменения движка;
3. гибридную модель данных: плоский список сегментов внутри pipeline для простоты обработки и иерархическую структуру встречи на уровне результата;
4. систему подключаемых модулей с реестром моделей, обеспечивающую замену любой модели через конфигурацию без модификации кода;
5. механизмы воспроизводимости (идентификатор запуска, сохранение конфигурации и промежуточных результатов) и отказоустойчивости (возобновление pipeline, комбинированная стратегия обработки ошибок);
6. событийную модель взаимодействия ядра с интерфейсом через шину событий и Qt-адаптер, обеспечивающую слабую связность и потокобезопасность.

Описанная архитектура удовлетворяет всем функциональным и нефункциональным требованиям, сформулированным в первой главе, и обеспечивает основу для реализации системы, описанной в третьей главе.

Список литературы

- [1] Alec Radford и др. “Robust Speech Recognition via Large-Scale Weak Supervision”. В: *Proceedings of the 40th International Conference on Machine Learning (ICML)*. Т. 202. Proceedings of Machine Learning Research. PMLR, 2023, с. 28492—28518.
- [2] Anmol Gulati и др. “Conformer: Convolution-augmented Transformer for Speech Recognition”. В: *Proc. Interspeech 2020*. 2020, с. 5036—5040. DOI: [10.21437/Interspeech.2020-3015](https://doi.org/10.21437/Interspeech.2020-3015).
- [3] Alex Graves и др. “Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks”. В: (2006), с. 369—376.
- [4] William Chan и др. “Listen, Attend and Spell: A Neural Network that Learns to Translate Speech to Text”. В: *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2016, с. 4960—4964.
- [5] Ashish Vaswani и др. “Attention Is All You Need”. В: *Advances in Neural Information Processing Systems (NeurIPS)*. Т. 30. 2017.
- [6] Alexei Baevski и др. “wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations”. В: *Advances in Neural Information Processing Systems* 33 (2020), с. 12449—12460.
- [7] Hervé Bredin. “pyannote.audio 2.1 speaker diarization pipeline: principle, benchmark, and recipe”. В: *Proc. Interspeech 2023*. 2023, с. 1983—1987. DOI: [10.21437/Interspeech.2023-105](https://doi.org/10.21437/Interspeech.2023-105).
- [8] Hervé Bredin и др. “pyannote.audio: neural building blocks for speaker diarization”. В: *ICASSP 2020, IEEE International Conference on Acoustics, Speech, and Signal Processing*. Barcelona, Spain, 2020.
- [9] Alexis Plaquet и Hervé Bredin. “Powerset multi-class cross entropy loss for neural speaker diarization”. В: *Proc. Interspeech 2023*. 2023.
- [10] Brecht Desplanques, Jenthe Thienpondt и Kris Demuynck. “ECAPA-TDNN: Emphasized Channel Attention, Propagation and Aggregation in TDNN Based Speaker Verification”. В: *Proc. Interspeech 2020*. 2020, с. 3830—3834. DOI: [10.21437/Interspeech.2020-2650](https://doi.org/10.21437/Interspeech.2020-2650).
- [11] Jörgen Valk и Tanel Alumäe. “VoxLingua107: a Dataset for Spoken Language Recognition”. В: *Proc. IEEE SLT Workshop*. 2021.

- [12] Erich Gamma и др. *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA: Addison-Wesley, 1995.
- [13] Robert C. Martin. *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.
- [14] Gregor Hohpe и Bobby Woolf. "Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions". B: Addison-Wesley, 2003.
- [15] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [16] Riverbank Computing. *PyQt6: Python bindings for Qt 6*. <https://www.riverbankcomputing.com/software/pyqt/>. Дата обращения: 07.04.2023.
- [17] Andrew C. Morris, Viktoria Maier и Phil Green. "From WER and RIL to MER and WIL: improved evaluation measures for connected speech recognition". B: *Proc. Interspeech* (2004), с. 2765—2768.
- [18] Tae Jin Park и др. "A Review of Speaker Diarization: Recent Advances with Deep Learning". B: т. 76. Elsevier, 2022, с. 101415.
- [19] Vassil Panayotov и др. "Librispeech: An ASR corpus based on public domain audio books". B: *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2015, с. 5206—5210.
- [20] Alexis Conneau и др. "FLEURS: Few-shot Learning Evaluation of Universal Representations of Speech". B: *arXiv preprint arXiv:2205.12446* (2022).
- [21] Mirco Ravanelli и др. *SpeechBrain: A General-Purpose Speech Toolkit*. 2021. arXiv: 2106.04624 [eess.AS].
- [22] SYSTRAN. *faster-whisper: Faster Whisper transcription with CTranslate2*. <https://github.com/SYSTRAN/faster-whisper>. Дата обращения: 15.02.2026. 2023.
- [23] Adam Paszke и др. *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. 2019. arXiv: 1912.01703.
- [24] Georgi Gerganov. *whisper.cpp: Port of OpenAI's Whisper model in C/C++*. <https://github.com/ggerganov/whisper.cpp>. Дата обращения: 03.03.2026. 2023.

- [25] Alpha Cephei. *Vosk: Offline Speech Recognition API*. <https://alphacephei.com/vosk/>. Дата обращения: 22.02.2026. 2023.
- [26] David Snyder и др. “X-Vectors: Robust DNN Embeddings for Speaker Recognition”. В: *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2018, с. 5329—5333.
- [27] D. Richard Hipp. *SQLite Documentation*. <https://www.sqlite.org/docs.html>. Дата обращения: 19.03.2026. 2023.
- [28] PyInstaller Development Team. *PyInstaller: Freeze Python applications*. <https://pyinstaller.org/>. Дата обращения: 12.03.2026. 2023.